2026

SOLUTION DESIGN

3:2 ANALYTICS

USE CASES

From all the use cases shown in the diagram, we picked the ones that actually matter the most to understand how the app works end to end. We didn't include stuff like Log out or Reset password because honestly, those are pretty self-explanatory and don't add much value describing them in detail.

The selected use cases are:

1. Register account
2. Add expense (manual)
3. Create monthly budget (by category)
4. Sync transactions
5. View dashboard / simple analytics

UC-01: REGISTER ACCOUNT

Field	Description
Name	Register account
Short description	A new user creates an account by entering their basic information so they can start using the app.
Precondition	The user does not have an existing account. The app is installed and there is internet connection.
Postcondition	The user account is created and stored in the system. The user is redirected to the main screen.
Error situations	The email entered is already registered. Required fields are left empty. Password doesn't meet minimum requirements.
System state on error	No account is created. The user stays on the registration screen with an error message shown.
Actors	User
Trigger	The user opens the app for the first time and taps "Create account".
Standard process	(1) User opens the app and selects "Register". (2) User enters name, email, and password. (3) System validates that the email is not already in use. (4) System validates that all fields are filled and the password meets the requirements. (5) System creates the account and stores the data. (6) System confirms registration and takes the user to the home screen.
Alternative processes	(3') The email is already registered → System shows an error and asks the user to log in or use a different email. (4') Password is too weak → System asks the user to enter a stronger password.

UC-02: ADD EXPENSE (MANUAL)

Field	Description
Name	Add expense (manual)
Short description	The user manually logs an expense by entering the amount, description, and date. The system automatically categorizes it.
Precondition	The user is logged in.
Postcondition	The expense is saved in the system and assigned to a category. The user's balance and history are updated.
Error situations	The user leaves the amount field empty. The user enters an invalid amount (negative number, letters, etc.).
System state on error	The expense is not saved. The user stays on the expense form with an error message.
Actors	User
Trigger	The user taps the "Add expense" button from the home screen or the expense section.
Standard process	(1) User selects "Add expense". (2) User enters amount, description, and date. (3) System validates the data. (4) System automatically categorizes the expense (<<include>> Categorize expense). (5) System saves the expense. (6) System shows confirmation and updates the expense list.
Alternative processes	(4') The user disagrees with the automatic category → User manually selects a different category before saving. (2') The user wants to add an expense without a description → System saves it with "No description" as default.

UC-03: CREATE MONTHLY BUDGET (BY CATEGORY)

Field	Description
Name	Create monthly budget (by category)
Short description	The user sets a spending limit for one or more categories for the current month.
Precondition	The user is logged in. There is no existing budget for the selected category in the current month.
Postcondition	The budget is created and stored. The system will now track spending against this limit.
Error situations	The user enters an invalid amount (zero, negative). The user tries to create a budget for a category that already has one this month.
System state on error	No budget is created. The user stays on the budget creation screen with the corresponding error message.
Actors	User
Trigger	The user navigates to the budget section and selects "Create budget".
Standard process	(1) User opens the budget section. (2) User selects a category (food, transport, entertainment, etc.). (3) User enters the monthly spending limit. (4) System validates that no budget exists for that category this month. (5) System saves the budget. (6) System shows the budget in the overview with a progress bar at 0%.

Alternative processes	(4') A budget already exists for that category → System asks if the user wants to edit the existing one instead. (3') User wants to set budgets for multiple categories at once → System allows adding more categories before saving.

UC-04: VIEW EXPENSE HISTORY

Field	Description
Name	View expense history
Short description	The user checks their past expenses, with the ability to filter by date, category, or source (manual vs bank).
Precondition	The user is logged in.
Postcondition	The system state doesn't change. The user sees a list of their recorded expenses.
Error situations	No expenses found for the selected filters.
System state on error	System remains unchanged. An empty state message is shown ("No expenses found").
Actors	User
Trigger	The user navigates to the expense history section.
Standard process	(1) User opens the expense history screen. (2) System loads and displays all expenses, sorted by most recent first. (3) User scrolls through the list to review their expenses.
Alternative processes	(2') User applies a filter (by date range, category, or source) → System shows only the expenses matching the filter. (2") There are no expenses at all → System shows an empty state with a suggestion to add one.

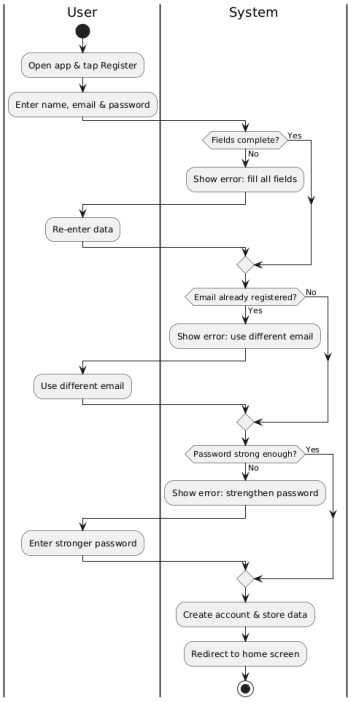
UC-05: VIEW DASHBOARD / SIMPLE ANALYTICS

Field	Description
Name	View dashboard / simple analytics
Short description	The user accesses a summary of their financial activity including spending by category, budget progress, and recent transactions.
Precondition	The user is logged in.
Postcondition	No data is changed. The user sees their financial overview.
Error situations	There's not enough data to generate meaningful charts (e.g., brand new account with no expenses).
System state on error	System remains unchanged. Dashboard shows placeholder content or tips to get started.
Actors	User
Trigger	The user opens the app (if dashboard is the home screen) or taps on "Dashboard" in the navigation.
Standard process	(1) User opens the dashboard. (2) System loads spending data, budget progress, and recent transactions. (3) System displays charts showing

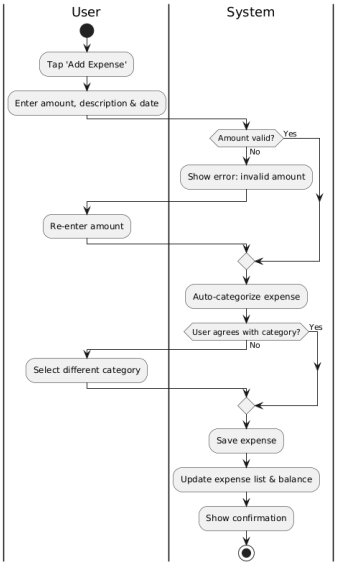
	spending by category, budget usage percentage, and a short list of the latest expenses. (4) User reviews the information.
Alternative processes	(2') The user has connected a bank account → Dashboard also shows synced transaction summaries alongside manual expenses. (3') The user taps on a specific category chart → System navigates to the filtered expense history for that category (this is the <<extend>> relationship from View financial tips).

ACTIVITY DIAGRAMS

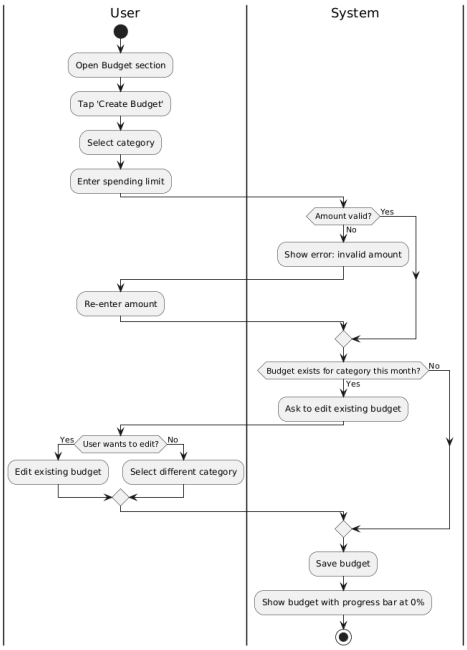
1. REGISTER ACCOUNT



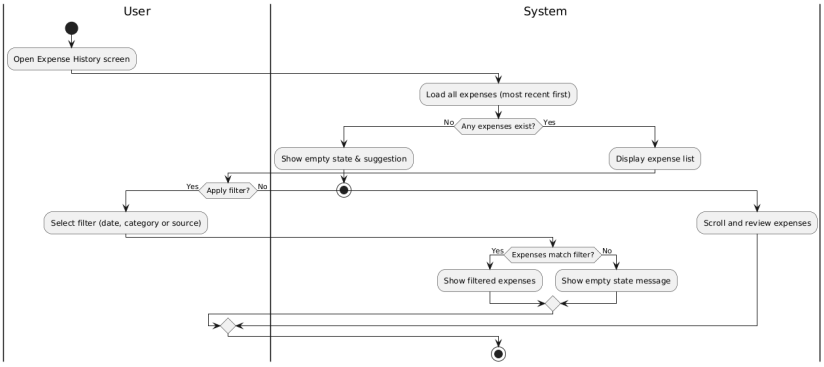
2. ADD EXPENSES



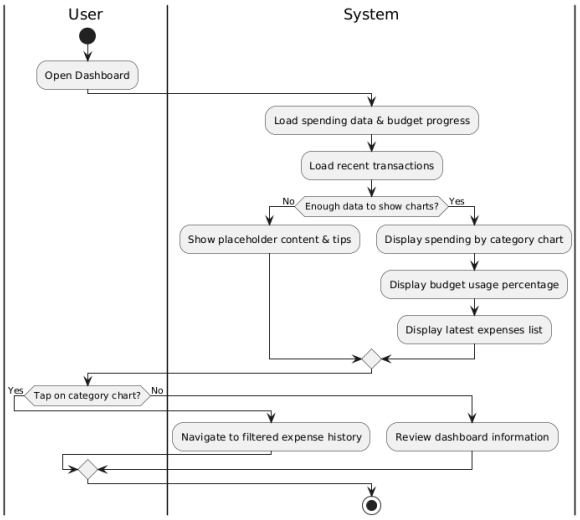
3. CREATE MONTHLY BUDGET



4. VIEW EXPENSES HISTORY



5. VIEW DASHBOARD



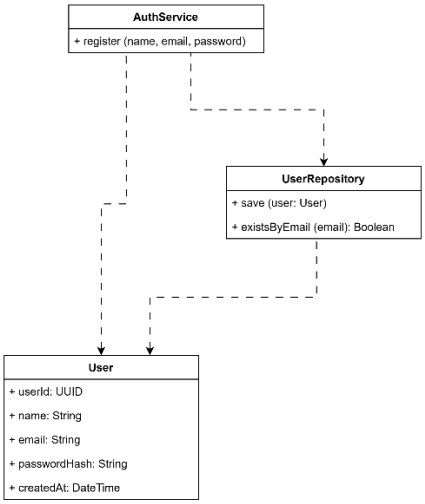
CLASS DIAGRAMS

Below we present the UML class diagrams developed for each use case in our MVP. Instead of showing one large global model, we provide a focused diagram per use case, including only the classes and relationships that participate in that specific functionality.

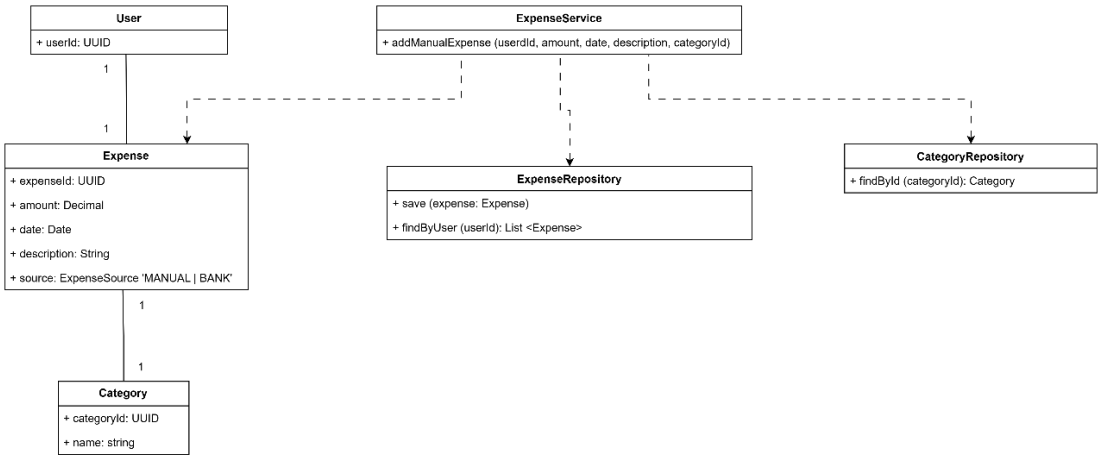
This makes the design easier to understand and ensures clear alignment with the project scope: account registration, manual expense creation, monthly budgets by category, viewing expense history, and a simple analytics dashboard.

Across the diagrams, the same core domain entities (User, Expense, Category, Budget, and BudgetItem) are reused to maintain consistency, while service and repository classes are included when they are necessary to represent the main responsibilities and data access for each use case.

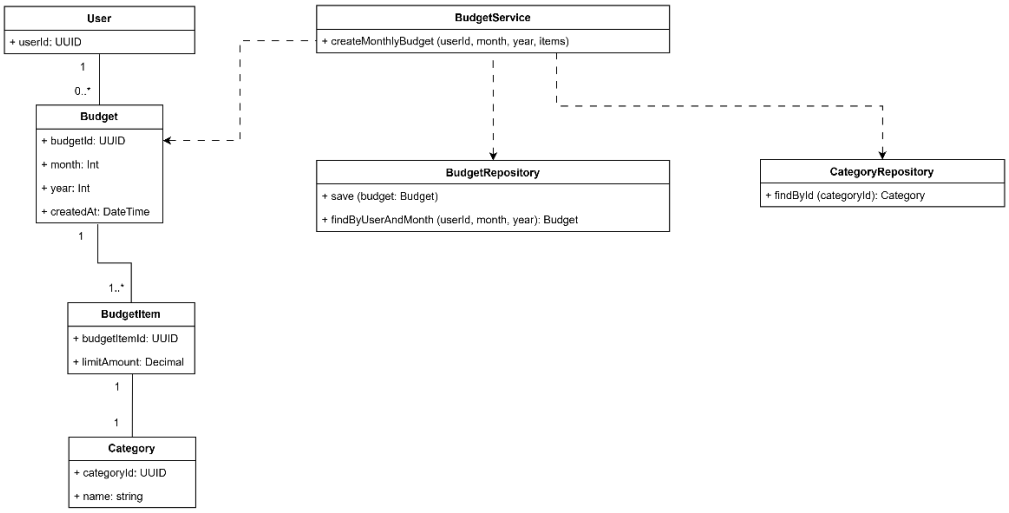
UC-01: REGISTER ACCOUNT



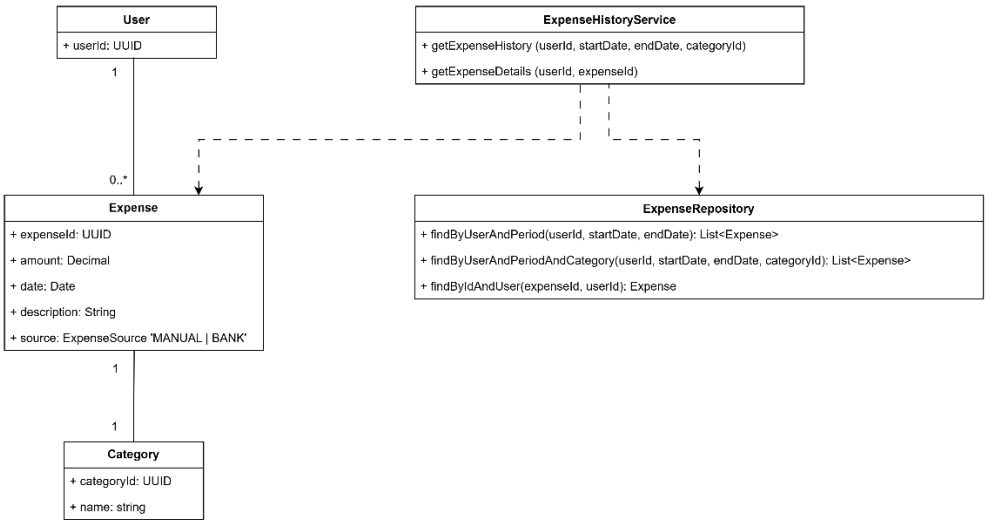
UC-02: ADD EXPENSE



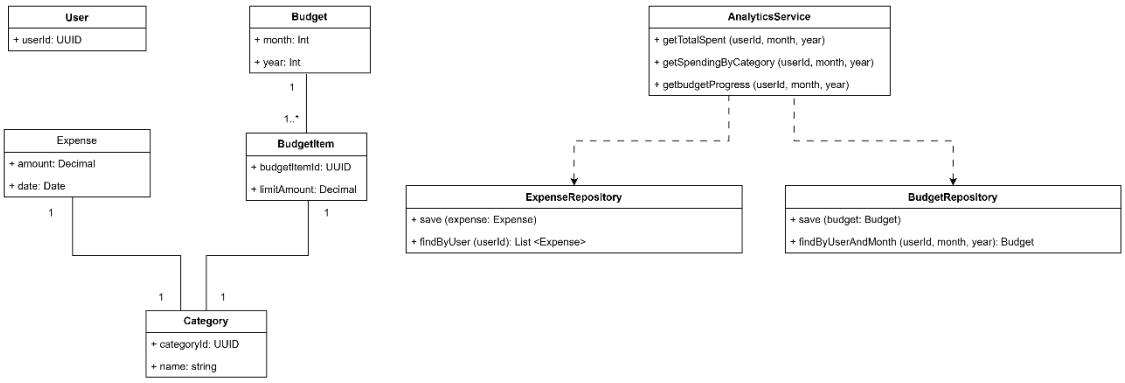
UC-03: CREATE MONTHLY BUDGET



UC-04: VIEW EXPENSE HISTORY



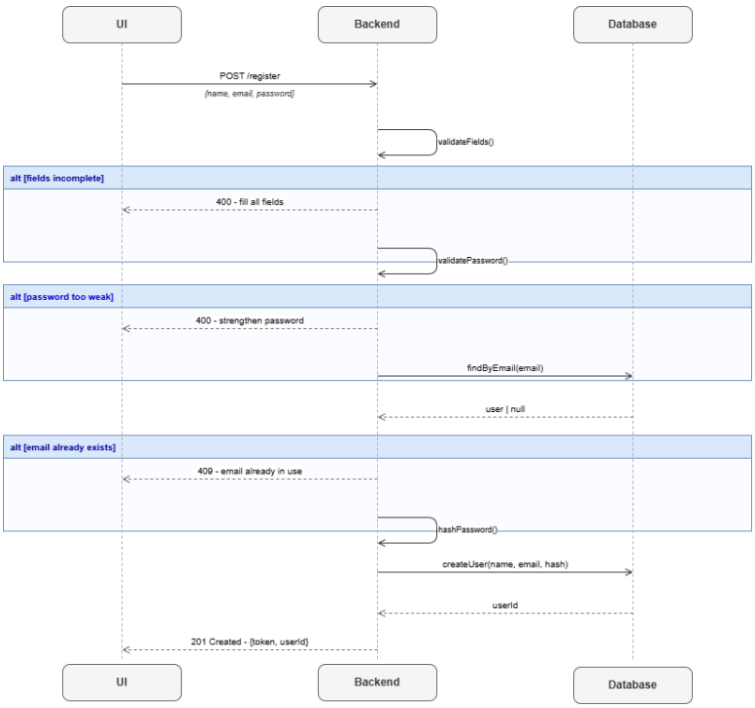
UC-05: VIEW DASHBOARD / SIMPLE ANALYTICS



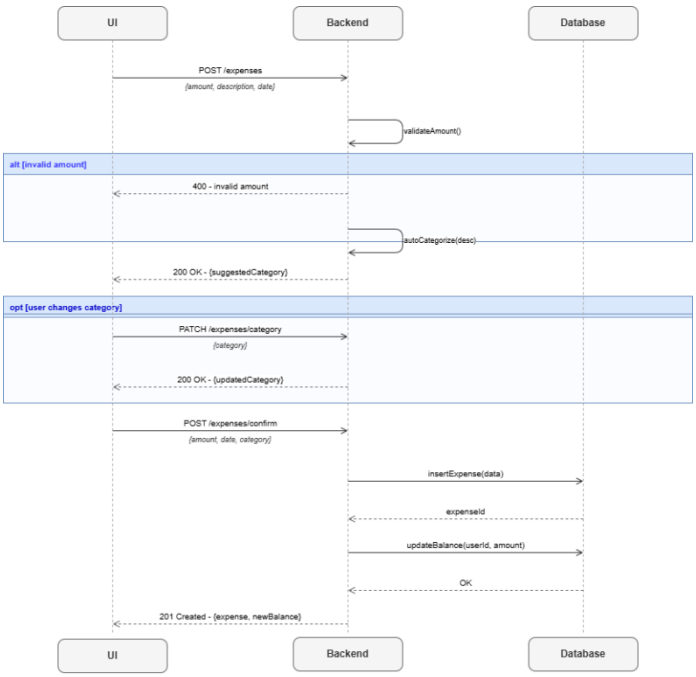
SEQUENCE DIAGRAMS

Sequence diagrams illustrate the interaction between the main components of the system — user interface, backend, and database — for each selected use case. They show how requests are initiated from the UI, processed by the backend, and supported by database operations, highlighting the order of messages, system responses, and alternative flows in case of errors or validation issues. These diagrams ensure a clear understanding of how data and control move through the application.

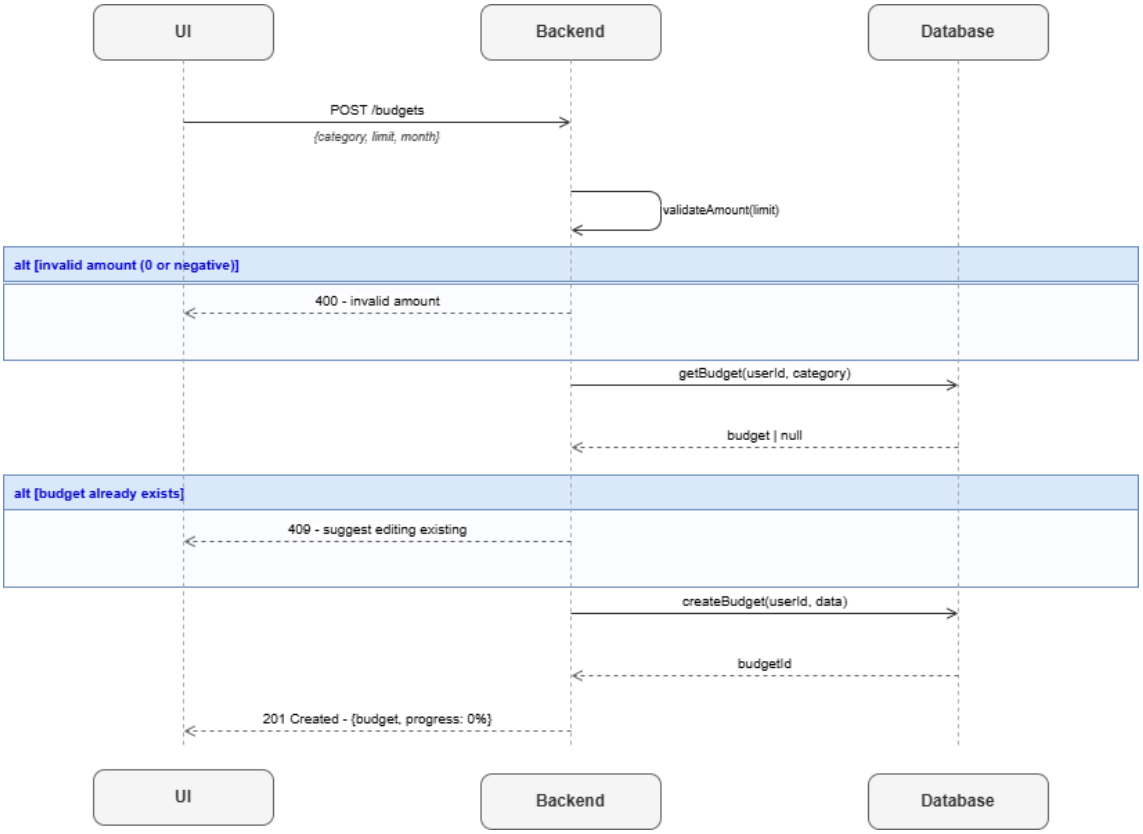
1. REGISTER ACCOUNT



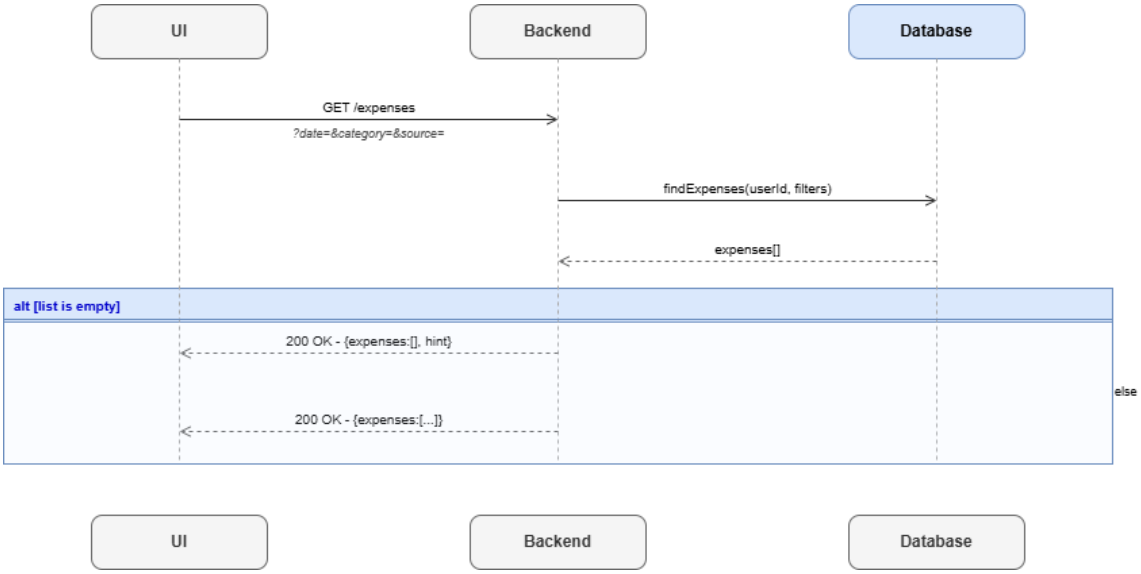
2. ADD EXPENSES



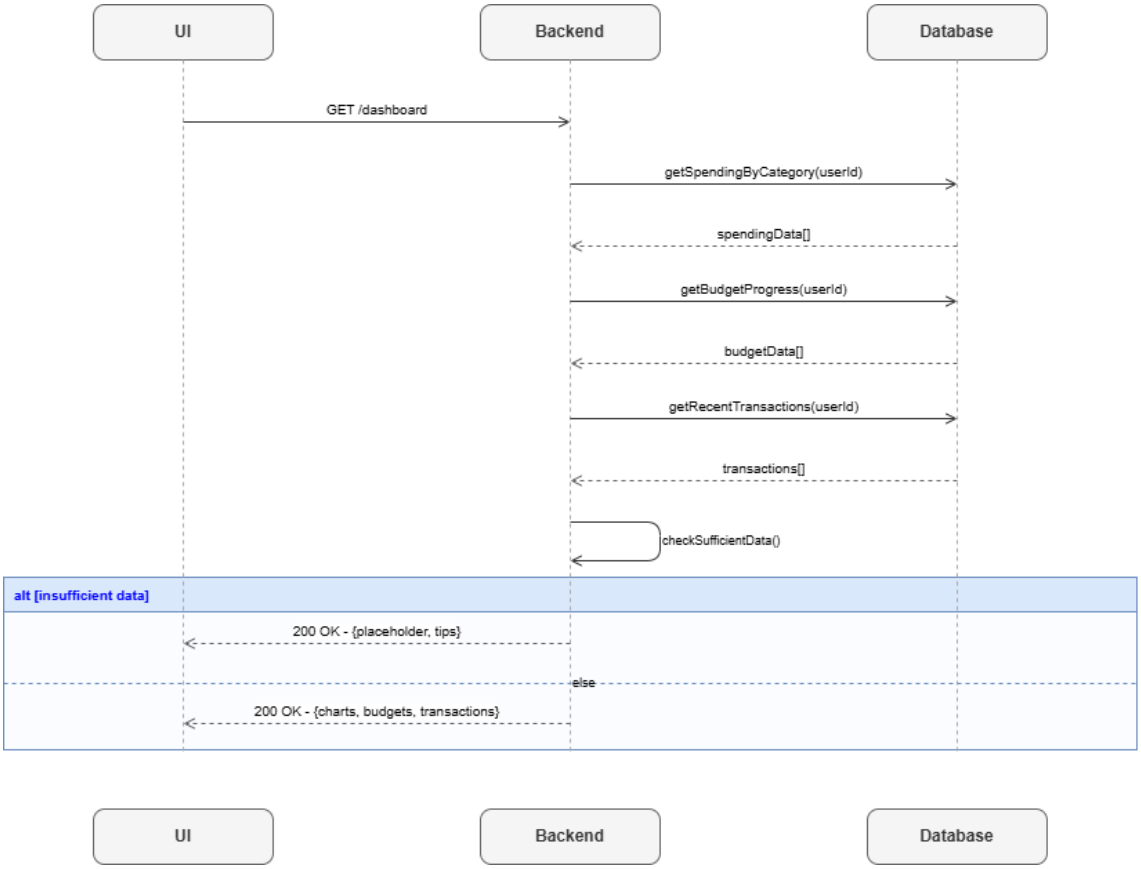
3. CREATE MONTHLY BUDGET



4. VIEW EXPENSES HISTORY



5. VIEW DASHBOARD



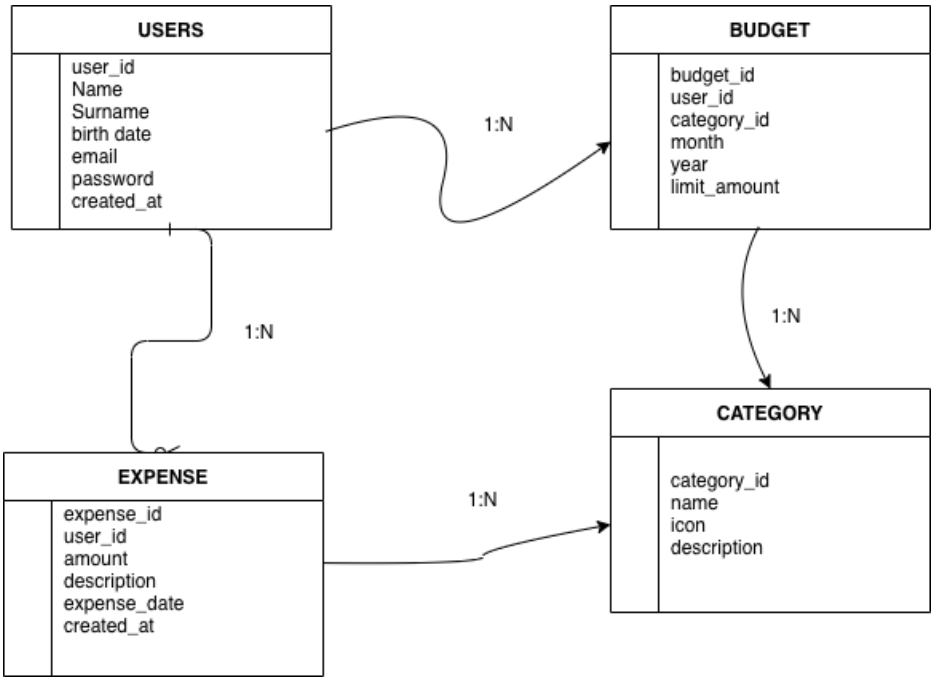
UI
Backend
Database

ENTITY RELATIONSHIP DIAGRAMS

The Entity-Relationship Diagram (ERD) represents the core data structure of the application. It includes four main entities: **Users, Expenses, Categories, and Budgets**.

A user can create multiple expenses and budgets, while each expense and budget is associated with a specific category. Categories are shared across the system to classify expenses and budgets.

The relationships between entities are defined as one-to-many (1:N), ensuring that each record is properly linked and organized. This design supports the main functionalities of the application, such as user registration, expense tracking, budget management, and data visualization.



DATABASE SCHEMA

The database schema defines the physical structure of the application's data based on the ERD. Each entity is implemented as a table with its corresponding attributes, data types, and constraints, including primary keys and foreign keys.

The schema consists of four main tables: **Users, Expenses, Categories, and Budgets**. Relationships between tables are implemented through foreign keys, ensuring data integrity and proper linkage between users, their expenses, and their budgets by category.

This structure supports the main functionalities of the system, enabling efficient data storage, retrieval, and analysis for expense tracking and budget management.

USERS

COLUMN	TYPE	CONSTRAINTS
USER_ID	INT	PRIMARY KEY
NAME	VARCHAR(50)	NOT NULL
SURNAME	VARCHAR(50)	NOT NULL
BIRTHDATE	DATE	NOT NULL
EMAIL	VARCHAR(100)	NOT NULL, UNIQUE
PASSWORD	VARCHAR(100)	NOT NULL
CREATED_AT	DATETIME	NOT NULL

CATEGORY

COLUMN	TYPE	CONSTRAINTS
CATEGORY_ID	INT	PRIMARY KEY
NAME	VARCHAR(50)	NOT NULL
ICON	VARCHAR(100)	
DESCRIPTION	VARCHAR(100)	

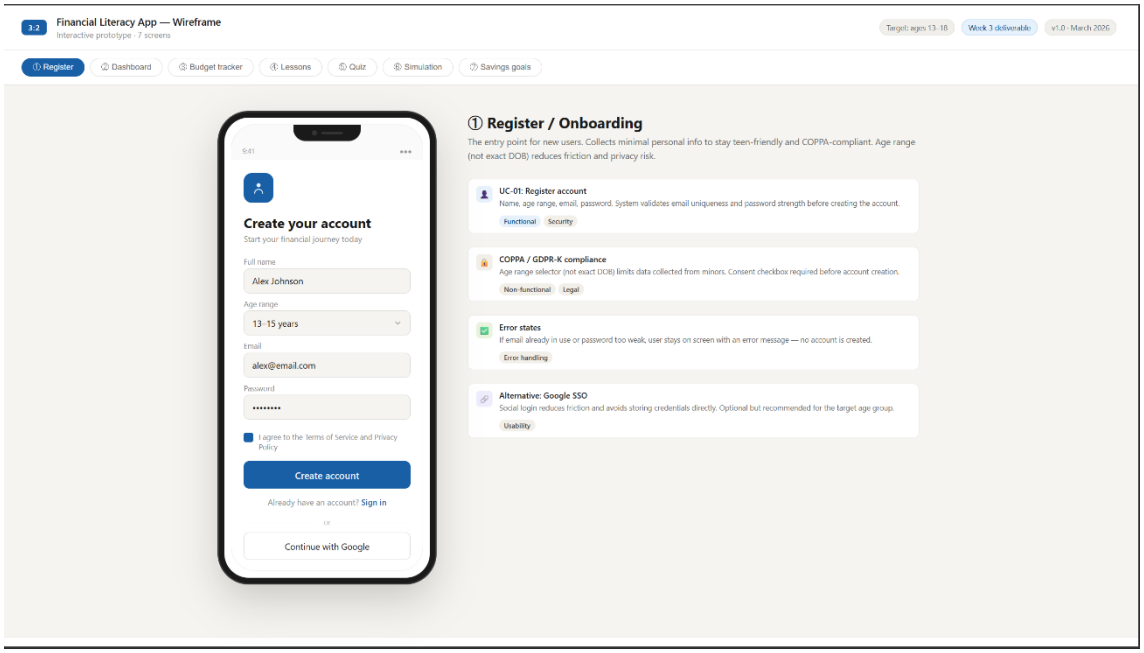
EXPENSE

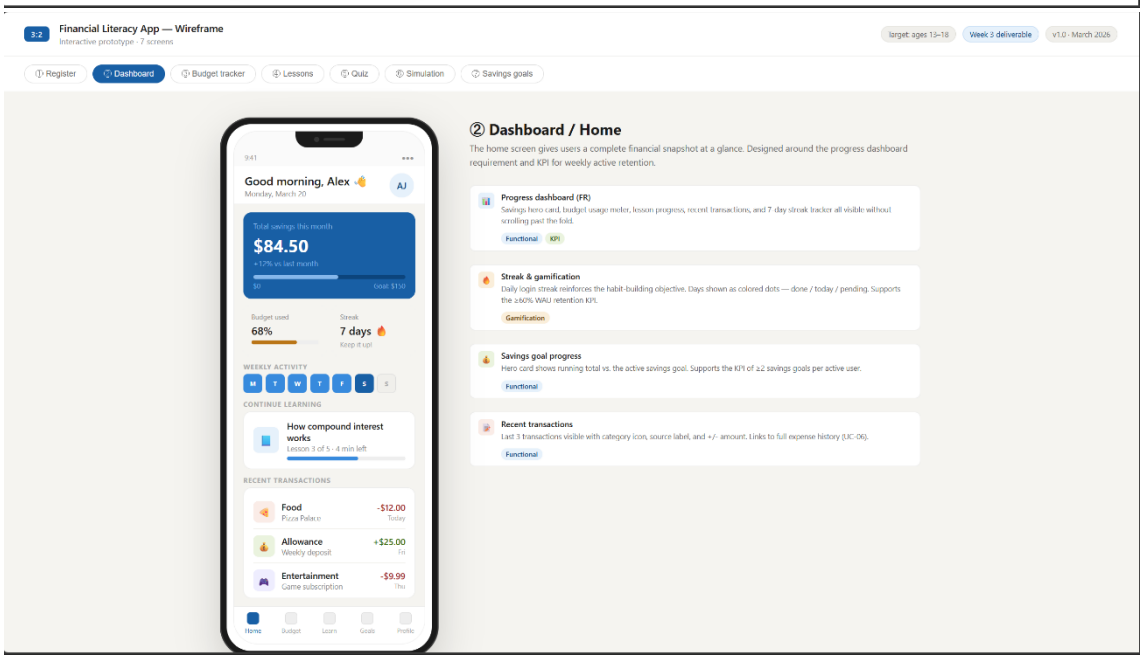
COLUMN	TYPE	CONSTRAINTS
EXPENSE_ID	INT	PRIMARY KEY
USER_ID	INT	FOREIGN KEY
CATEGORY_ID	INT	FOREIGN KEY
AMOUNT	DECIMAL	NOT NULL
DESCRIPTION	VARCHAR(255)	NOT NULL
EXPENSE_DATE	DATE	NOT NULL
CREATED_AT	DATETIME	NOT NULL

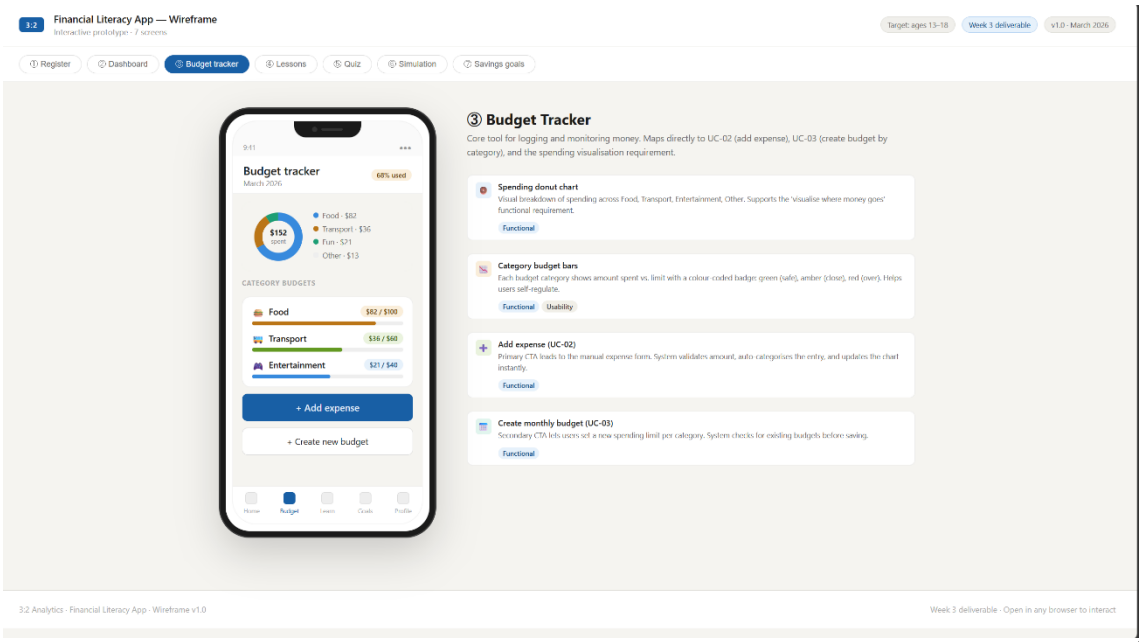
CATEGORY

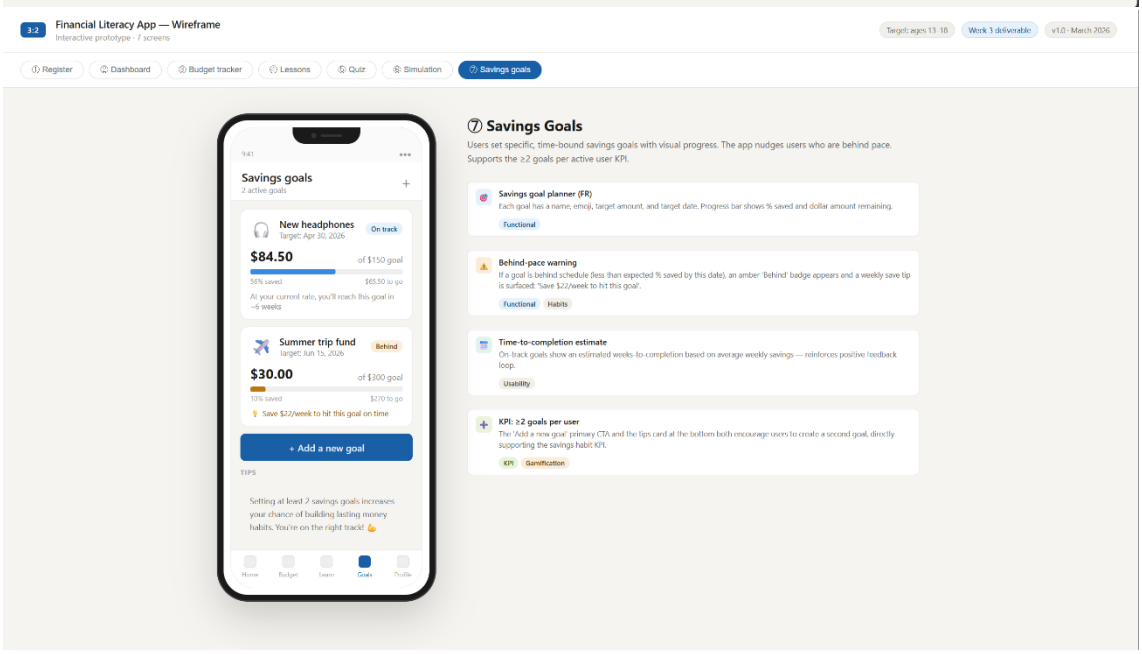
COLUMN	TYPE	CONSTRAINTS
BUDGET_ID	INT	PRIMARY KEY
USER_ID	INT	FOREIGN KEY
CATEGORY_ID	INT	FOREIGN KEY
MONTH	INT	NOT NULL
YEAR	INT	NOT NULL
LIMIT_AMOUNT	DECIMAL	NOT NULL
CREATED_AT	DATETIME	NOT NULL

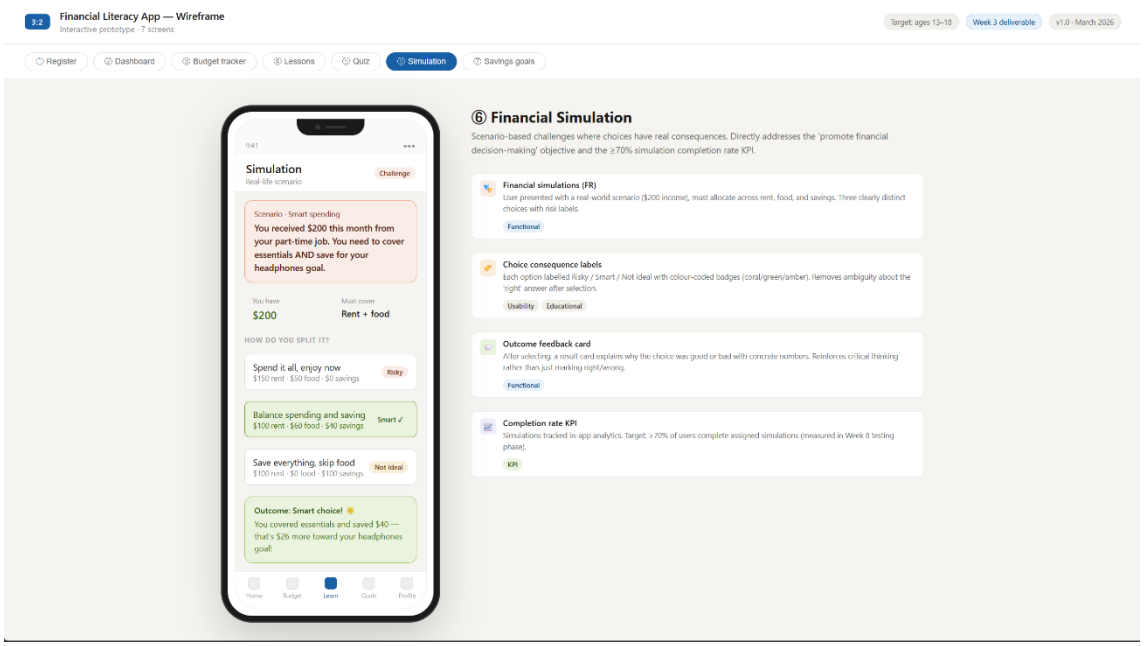
WIREFRAME

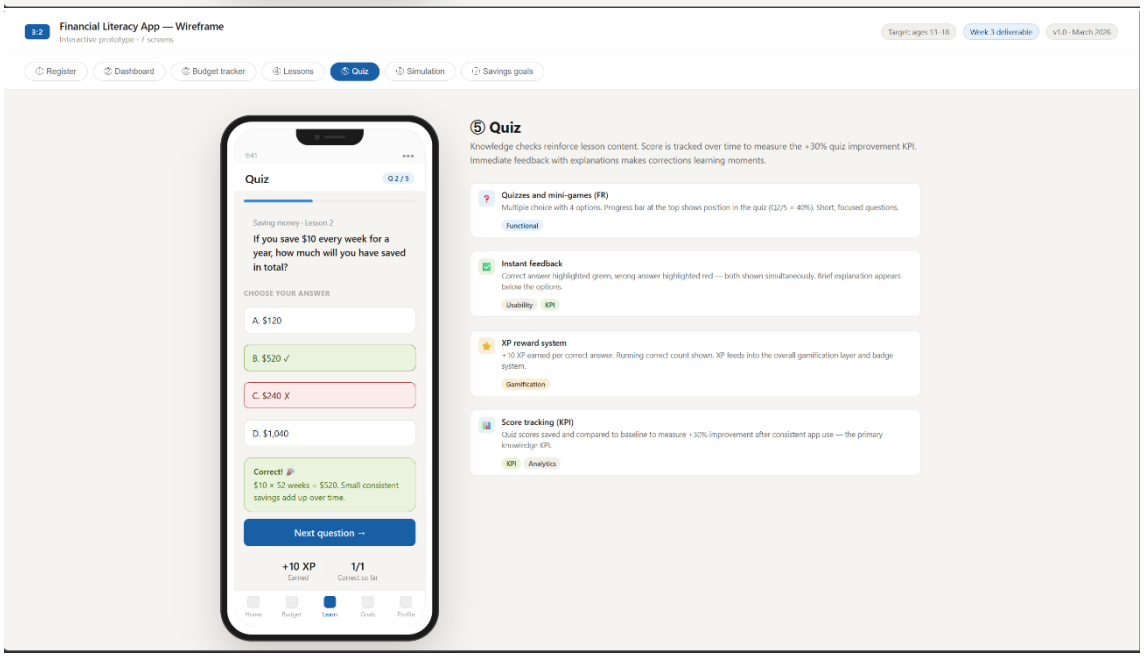

3:2
Financial Literacy App — Wireframe
Interactive prototype · 7 screens
Target: ages 13–18
Week 3 deliverable
v1.0 · March 2026
① Register
② Dashboard
③ Budget tracker
④ Lessons
⑤ Quiz
⑥ Simulation
⑦ Savings goals
9:41
Create your account
Start your financial journey today
Full name
Alex Johnson
Age range
13–15 years
Email
alex@email.com
Password
I agree to the Terms of Service and Privacy Policy
Create account
Already have an account? Sign in
or
Continue with Google
① Register / Onboarding
The entry point for new users. Collects minimal personal info to stay teen-friendly and COPPA-compliant. Age range (not exact DOB) reduces friction and privacy risk.
UC-01: Register account
Name, age range, email, password. System validates email uniqueness and password strength before creating the account.
Functional
Security
COPPA / GDPR-K compliance
Age range selector (not exact DOB) limits data collected from minors. Consent checkbox required before account creation.
Non-functional
Legal
Error states
If email already in use or password too weak, user stays on screen with an error message — no account is created.
Error handling
Alternative: Google SSO
Social login reduces friction and avoids storing credentials directly. Optional but recommended for the target age group.
Usability

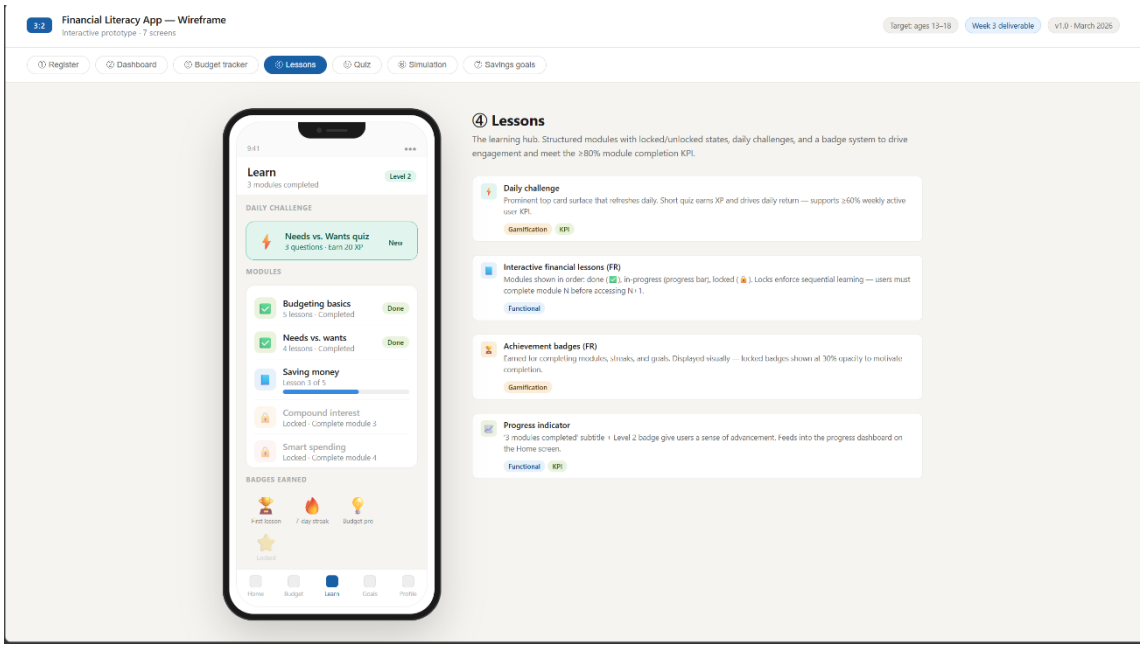

3:2
Financial Literacy App — Wireframe
Interactive prototype · 7 screens
target: ages 13–18
Week 3 deliverable
v1.0 · March 2026
① Register
② Dashboard
③ Budget tracker
④ Lessons
⑤ Quiz
⑥ Simulation
⑦ Savings goals
9:41
Good morning, Alex 👋
Monday, March 20
AJ
Total savings this month
$84.50
+17% vs last month
$0
Goal $150
Budget used
68%
Streak
7 days 🔥
Keep it up!
WEEKLY ACTIVITY
M T W T F S S
CONTINUE LEARNING
How compound interest works
Lesson 3 of 5 · 4 min left
RECENT TRANSACTIONS
Food
Pizza Palace
-$12.00
Today
Allowance
Weekly deposit
+$25.00
Fri
Entertainment
Game subscription
-$9.99
Thu
Home
Budget
Learn
Goals
Profile
② Dashboard / Home
The home screen gives users a complete financial snapshot at a glance. Designed around the progress dashboard requirement and KPI for weekly active retention.
Progress dashboard (FR)
Savings hero card, budget usage meter, lesson progress, recent transactions, and 7 day streak tracker all visible without scrolling past the fold.
Functional
KPI
Streak & gamification
Daily login streak reinforces the habit-building objective. Days shown as colored dots — done / today / pending. Supports the ≥60% WAU retention KPI.
Gamification
Savings goal progress
Hero card shows running total vs. the active savings goal. Supports the KPI of ≥2 savings goals per active user.
Functional
Recent transactions
Last 3 transactions visible with category icon, source label, and +/- amount. Links to full expense history (UC-06).
Functional


3:2
Financial Literacy App — Wireframe
Interactive prototype · 7 screens
Target: ages 13–18
Week 3 deliverable
v1.0 · March 2026
① Register
② Dashboard
③ Budget tracker
④ Lessons
⑤ Quiz
⑥ Simulation
⑦ Savings goals
9:41
Budget tracker
March 2026
68% used
$152
spent
Food · $82
Transport · $36
Fun · $21
Other · $13
CATEGORY BUDGETS
Food
$82 / $100
Transport
$36 / $60
Entertainment
$21 / $40
+ Add expense
+ Create new budget
Home
Budget
Learn
Goals
Profile
③ Budget Tracker
Core tool for logging and monitoring money. Maps directly to UC-02 (add expense), UC-03 (create budget by category), and the spending visualisation requirement.
Spending donut chart
Visual breakdown of spending across Food, Transport, Entertainment, Other. Supports the 'visualise where money goes' functional requirement.
Functional
Category budget bars
Each budget category shows amount spent vs. limit with a colour-coded badge: green (safe), amber (close), red (over). Helps users self-regulate.
Functional
Usability
Add expense (UC-02)
Primary CTA leads to the manual expense form. System validates amount, auto-categorises the entry, and updates the chart instantly.
Functional
Create monthly budget (UC-03)
Secondary CTA lets users set a new spending limit per category. System checks for existing budgets before saving.
Functional
3:2 Analytics · Financial Literacy App · Wireframe v1.0
Week 3 deliverable · Open in any browser to interact


3:2
Financial Literacy App — Wireframe
Interactive prototype · 7 screens
Target: ages 13–18
Week 3 deliverable
v1.0 · March 2026
① Register
② Dashboard
③ Budget tracker
④ Lessons
⑤ Quiz
⑥ Simulation
⑦ Savings goals
9:41
Savings goals
2 active goals
New headphones
Target: Apr 30, 2026
On track
$84.50
of $150 goal
56% saved
$65.50 to go
At your current rate, you'll reach this goal in ~6 weeks
Summer trip fund
Target: Jun 15, 2026
Behind
$30.00
of $300 goal
10% saved
$270 to go
Save $22/week to hit this goal on time
+ Add a new goal
TIPS
Setting at least 2 savings goals increases your chance of building lasting money habits. You're on the right track!
Home
Budget
Learn
Goals
Profile
⑦ Savings Goals
Users set specific, time-bound savings goals with visual progress. The app nudges users who are behind pace. Supports the ≥2 goals per active user KPI.
Savings goal planner (FR)
Each goal has a name, emoji, target amount, and target date. Progress bar shows % saved and dollar amount remaining.
Functional
Behind-pace warning
If a goal is behind schedule (less than expected % saved by this date), an amber 'Behind' badge appears and a weekly save tip is surfaced: 'Save $22/week to hit this goal'.
Functional
Habits
Time-to-completion estimate
On-track goals show an estimated weeks-to-completion based on average weekly savings — reinforces positive feedback loop.
Usability
KPI: ≥2 goals per user
The 'Add a new goal' primary CTA and the tips card at the bottom both encourage users to create a second goal, directly supporting the savings habit KPI.
KPI
Gamification


3:2
Financial Literacy App — Wireframe
Interactive prototype · 7 screens
Target: ages 13–18
Week 3 deliverable
v1.0 · March 2026
Register
Dashboard
Budget tracker
Lessons
Quiz
Simulation
Savings goals
Simulation
Real-life scenario
Challenge
Scenario · Smart spending
You received $200 this month from your part-time job. You need to cover essentials AND save for your headphones goal.
You have
$200
Must cover
Rent + food
HOW DO YOU SPLIT IT?
Spend it all, enjoy now
$150 rent · $50 food · $0 savings
Risky
Balance spending and saving
$100 rent · $60 food · $40 savings
Smart ✓
Save everything, skip food
$100 rent · $0 food · $100 savings
Not ideal
Outcome: Smart choice!
You covered essentials and saved $40 — that's $26 more toward your headphones goal!
Home
Budget
Learn
Goals
Profile
⑥ Financial Simulation
Scenario-based challenges where choices have real consequences. Directly addresses the 'promote financial decision-making' objective and the ≥70% simulation completion rate KPI.
Financial simulations (FR)
User presented with a real-world scenario ($200 income), must allocate across rent, food, and savings. Three clearly distinct choices with risk labels.
Functional
Choice consequence labels
Each option labelled Risky / Smart / Not ideal with colour-coded badges (coral/green/amber). Removes ambiguity about the 'right' answer after selection.
Usability
Educational
Outcome feedback card
After selecting, a result card explains why the choice was good or bad with concrete numbers. Reinforces critical thinking rather than just marking right/wrong.
Functional
Completion rate KPI
Simulations tracked in-app analytics. Target: ≥70% of users complete assigned simulations (measured in Week 8 testing phase).
KPI


3:2
Financial Literacy App — Wireframe
Interactive prototype · 7 screens
Target: ages 13–18
Week 3 deliverable
v1.0 · March 2026
Register
Dashboard
Budget tracker
Lessons
Quiz
Simulation
Savings goals
Quiz
Q 2 / 5
Saving money · Lesson 2
If you save $10 every week for a year, how much will you have saved in total?
CHOOSE YOUR ANSWER
A. $120
B. $520 ✓
C. $240 ✗
D. $1,040
Correct! 🎉
$10 × 52 weeks = $520. Small consistent savings add up over time.
Next question →
+10 XP
Earned
1/1
Correct so far
Home
Budget
Learn
Goals
Profile
⑤ Quiz
Knowledge checks reinforce lesson content. Score is tracked over time to measure the +30% quiz improvement KPI. Immediate feedback with explanations makes corrections learning moments.
Quizzes and mini-games (FR)
Multiple choice with 4 options. Progress bar at the top shows position in the quiz (Q2/5 = 40%). Short, focused questions.
Functional
Instant feedback
Correct answer highlighted green, wrong answer highlighted red — both shown simultaneously. Brief explanation appears below the options.
Usability
KPI
XP reward system
+10 XP earned per correct answer. Running correct count shown. XP feeds into the overall gamification layer and badge system.
Gamification
Score tracking (KPI)
Quiz scores saved and compared to baseline to measure +30% improvement after consistent app use — the primary knowledge KPI.
KPI
Analytics


3:2
Financial Literacy App — Wireframe
Interactive prototype · 7 screens
Target: ages 13–18
Week 3 deliverable
v1.0 · March 2026
Register
Dashboard
Budget tracker
Lessons
Quiz
Simulation
Savings goals
9:41
Learn
3 modules completed
Level 2
DAILY CHALLENGE
Needs vs. Wants quiz
3 questions · Earn 20 XP
New
MODULES
Budgeting basics
5 lessons · Completed
Done
Needs vs. wants
4 lessons · Completed
Done
Saving money
Lesson 3 of 5
Compound interest
Locked · Complete module 3
Smart spending
Locked · Complete module 4
BADGES EARNED
First lesson
7 day streak
Budget pro
Locked
Home
Budget
Learn
Goals
Profile
④ Lessons
The learning hub. Structured modules with locked/unlocked states, daily challenges, and a badge system to drive engagement and meet the ≥80% module completion KPI.
Daily challenge
Prominent top card surface that refreshes daily. Short quiz earns XP and drives daily return — supports ≥60% weekly active user KPI.
Gamification
KPI
Interactive financial lessons (FR)
Modules shown in order: done ( ), in-progress (progress bar), locked ( ). Locks enforce sequential learning — users must complete module N before accessing N+1.
Functional
Achievement badges (FR)
Earned for completing modules, streaks, and goals. Displayed visually — locked badges shown at 30% opacity to motivate completion.
Gamification
Progress indicator
'3 modules completed' subtitle + Level 2 badge give users a sense of advancement. Feeds into the progress dashboard on the Home screen.
Functional
KPI